

## ASSIGNMENT-16.1

NAME: SRAVANI ANUMALA

ROLL NO: 2303A510G7

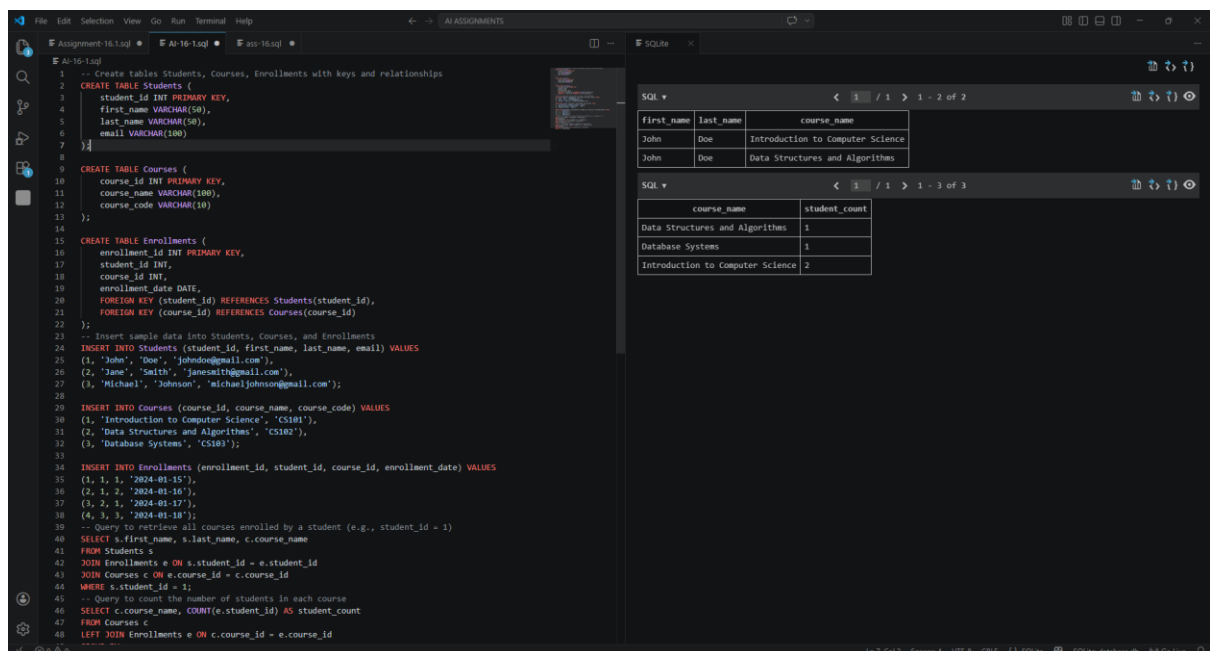
BATCH: 30

### TASK-1:

#### PROMPT:

--create tables Students, Courses, Enrollments, primary keys, foreign keys, and relationships

#### CODE:



```
1 -- Create tables Students, Courses, Enrollments with keys and relationships
2 CREATE TABLE Students (
3     student_id INT PRIMARY KEY,
4     first_name VARCHAR(50),
5     last_name VARCHAR(50),
6     email VARCHAR(100)
7 );
8
9 CREATE TABLE Courses (
10     course_id INT PRIMARY KEY,
11     course_name VARCHAR(100),
12     course_code VARCHAR(10)
13 );
14
15 CREATE TABLE Enrollments (
16     enrollment_id INT PRIMARY KEY,
17     student_id INT,
18     course_id INT,
19     enrollment_date DATE,
20     FOREIGN KEY (student_id) REFERENCES Students(student_id),
21     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
22 );
23
24 -- Insert sample data into Students, Courses, and Enrollments
25 INSERT INTO Students (student_id, first_name, last_name, email) VALUES
26 (1, 'John', 'Doe', 'john.doe@gmail.com'),
27 (2, 'Jane', 'Smith', 'janesmith@gmail.com'),
28 (3, 'Michael', 'Johnson', 'michaeljohnson@gmail.com');
29
30 INSERT INTO Courses (course_id, course_name, course_code) VALUES
31 (1, 'Introduction to Computer Science', 'CS101'),
32 (2, 'Data Structures and Algorithms', 'CS102'),
33 (3, 'Database Systems', 'CS103');
34
35 INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date) VALUES
36 (1, 1, 1, '2024-01-15'),
37 (2, 1, 2, '2024-01-16'),
38 (3, 2, 1, '2024-01-17'),
39 (4, 3, 3, '2024-01-18');
40
41 -- Query to retrieve all courses enrolled by a student (e.g., student_id = 1)
42 SELECT e.first_name, s.last_name, c.course_name
43 FROM Students s
44 JOIN Enrollments e ON s.student_id = e.student_id
45 JOIN Courses c ON e.course_id = c.course_id
46 WHERE s.student_id = 1;
47
48 -- Query to count the number of students in each course
49 SELECT c.course_name, COUNT(e.student_id) AS student_count
50 FROM Courses c
51 LEFT JOIN Enrollments e ON c.course_id = e.course_id
```

| first_name | last_name | course_name                      |
|------------|-----------|----------------------------------|
| John       | Doe       | Introduction to Computer Science |
| John       | Doe       | Data Structures and Algorithms   |

| course_name                      | student_count |
|----------------------------------|---------------|
| Data Structures and Algorithms   | 1             |
| Database Systems                 | 1             |
| Introduction to Computer Science | 2             |

#### OBSERVATION:

This task helped me understand how student and course data are stored in a database. I learned how to connect tables using primary key and foreign key. It also improved my knowledge of writing SQL queries to get useful information.

### TASK-2:

#### PROMPT:

--create tables doctors, patients, appointments, define constraints(unique IDs,valid dates)

#### CODE:

Assignment-16.1.sql AI-16-1.sql as-16.sql SQuire

```
36 (1, 'John', 'Doe', '1985-05-15'),
37 (2, 'Jane', 'Smith', '1990-08-22'),
38 (3, 'Michael', 'Williams', '1975-12-10');
39
40 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
41 (1, 1, 1, '2023-06-01'),
42 (2, 2, 2, '2023-06-02'),
43 (3, 3, 3, '2023-06-03');
44
45 -- Get all appointments for a specific doctor
46 SELECT d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
47 FROM Doctors d
48 JOIN Appointments a ON d.doctor_id = a.doctor_id
49 JOIN Patients p ON a.patient_id = p.patient_id
50 WHERE d.doctor_id = 1;
51
52 -- List patient history by patient ID
53 SELECT p.first_name, p.last_name, a.appointment_date, d.first_name, d.last_name, d.specialty
54 FROM Patients p
55 JOIN Appointments a ON p.patient_id = a.patient_id
56 JOIN Doctors d ON a.doctor_id = d.doctor_id
57 WHERE p.patient_id = 2;
58
59 -- Count total patients treated by each doctor
60 SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
61 FROM Doctors d
62 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
63 GROUP BY d.doctor_id;
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

| first_name | last_name | first_name | last_name | appointment_date |
|------------|-----------|------------|-----------|------------------|
| Dr. Smith  | Johnson   | John       | Doe       | 2023-06-01       |

Ln 45, Col 1 (264 selected) | Copied | UTF-8 | CRLF | SQuire | SQuire database.db | F4 Go Live

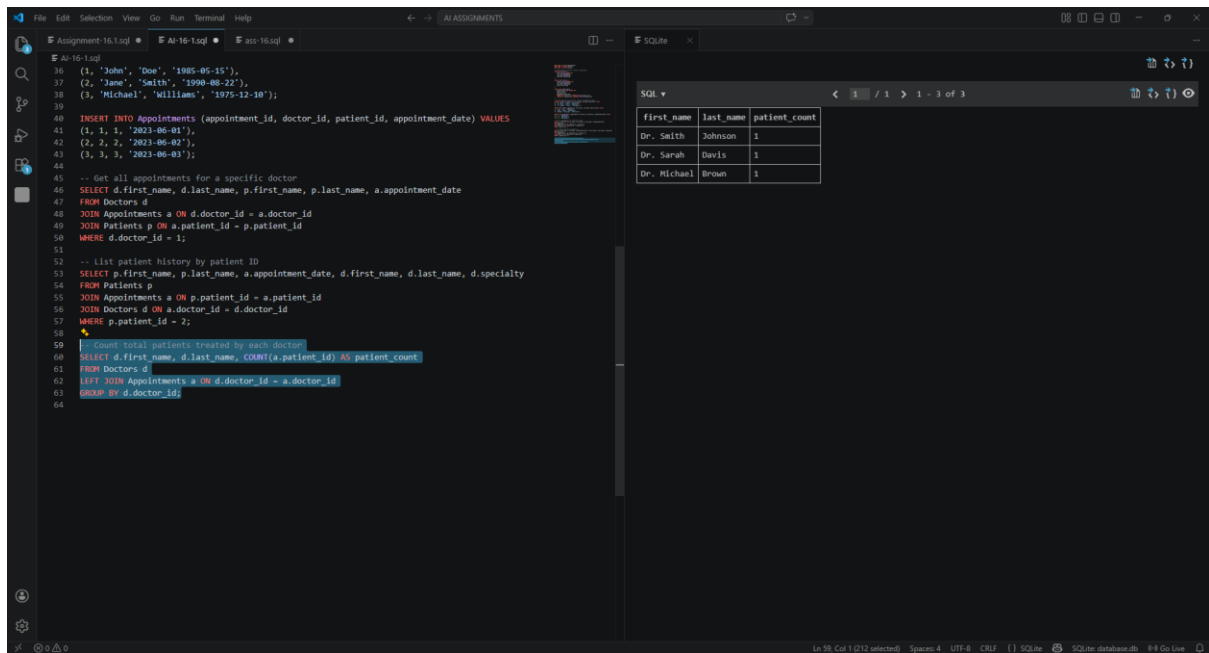
Assignment-16.1.sql AI-16-1.sql as-16.sql SQuire

```
36 (1, 'John', 'Doe', '1985-05-15'),
37 (2, 'Jane', 'Smith', '1990-08-22'),
38 (3, 'Michael', 'Williams', '1975-12-10');
39
40 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
41 (1, 1, 1, '2023-06-01'),
42 (2, 2, 2, '2023-06-02'),
43 (3, 3, 3, '2023-06-03');
44
45 -- Get all appointments for a specific doctor
46 SELECT d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
47 FROM Doctors d
48 JOIN Appointments a ON d.doctor_id = a.doctor_id
49 JOIN Patients p ON a.patient_id = p.patient_id
50 WHERE d.doctor_id = 1;
51
52 -- List patient history by patient ID
53 SELECT p.first_name, p.last_name, a.appointment_date, d.first_name, d.last_name, d.specialty
54 FROM Patients p
55 JOIN Appointments a ON p.patient_id = a.patient_id
56 JOIN Doctors d ON a.doctor_id = d.doctor_id
57 WHERE p.patient_id = 2;
58
59 -- Count total patients treated by each doctor
60 SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
61 FROM Doctors d
62 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
63 GROUP BY d.doctor_id;
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

| first_name | last_name | appointment_date | first_name | last_name | specialty  |
|------------|-----------|------------------|------------|-----------|------------|
| Jane       | Smith     | 2023-06-02       | Dr. Sarah  | Davis     | Pediatrics |

Ln 52, Col 1 (270 selected) | Copied | UTF-8 | CRLF | SQuire | SQuire database.db | F4 Go Live



```
File Edit Selection View Go Run Terminal Help
-- Assignment-16.1.sql -- AI-16.1.sql -- ass-16.sql --
26 (1, 'John', 'Doe', '1985-05-15'),
27 (2, 'Jane', 'Smith', '1990-08-22'),
28 (3, 'Michael', 'Williams', '1975-12-10');
29
30 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
41 (1, 1, 1, '2023-06-01'),
42 (2, 2, 2, '2023-06-02'),
43 (3, 3, 3, '2023-06-03');
44
45 -- Get all appointments for a specific doctor
46 SELECT d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
47 FROM Doctors d
48 JOIN Appointments a ON d.doctor_id = a.doctor_id
49 JOIN Patients p ON a.patient_id = p.patient_id
50 WHERE d.doctor_id = 1;
51
52 -- List patient history by patient ID
53 SELECT p.first_name, p.last_name, a.appointment_date, d.first_name, d.last_name, d.specialty
54 FROM Patients p
55 JOIN Appointments a ON p.patient_id = a.patient_id
56 JOIN Doctors d ON a.doctor_id = d.doctor_id
57 WHERE p.patient_id = 2;
58
59 -- Count total patients treated by each doctor
60 SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
61 FROM Doctors d
62 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
63 GROUP BY d.doctor_id;
```

| first_name  | last_name | patient_count |
|-------------|-----------|---------------|
| Dr. Smith   | Johnson   | 1             |
| Dr. Sarah   | Owens     | 1             |
| Dr. Michael | Brown     | 1             |

## OBSERVATION:

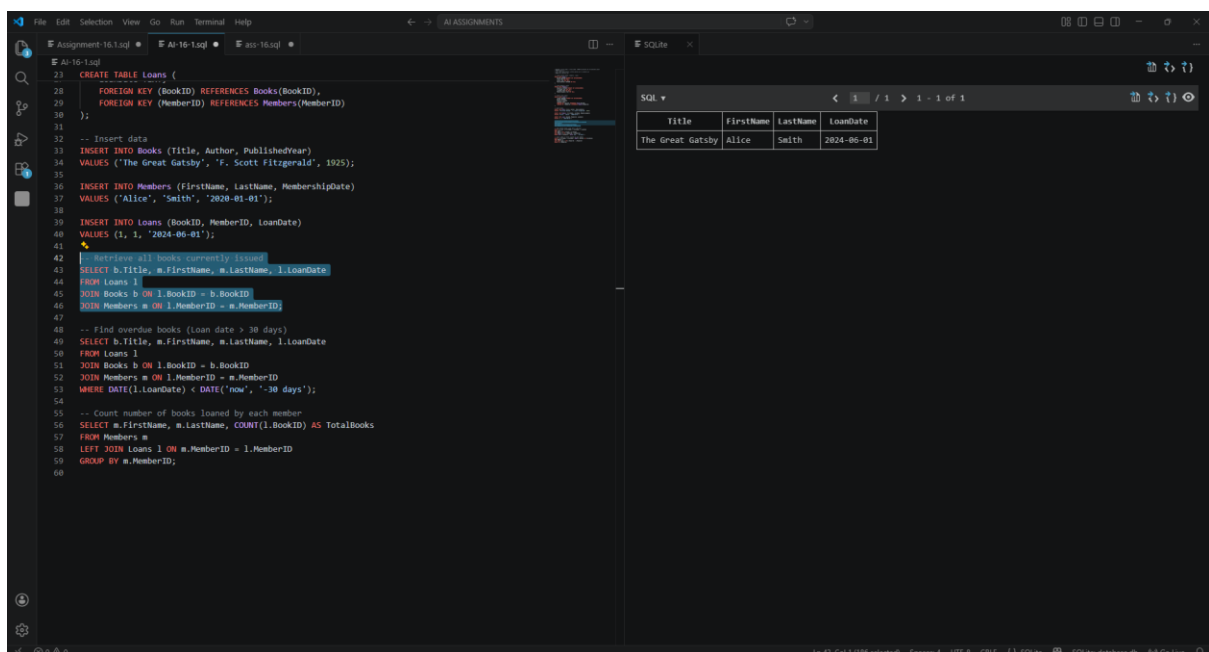
In this task, I understood how databases are used in hospitals to manage doctors, patients, and appointments. I learned about using constraints to avoid wrong data. It also helped me practice queries for real-life situations.

## TASK-3:

### PROMPT:

--create tables books,members,loans

### CODE:



```
File Edit Selection View Go Run Terminal Help
-- Assignment-16.1.sql -- AI-16.1.sql -- ass-16.sql --
23 CREATE TABLE Loans (
24     FOREIGN KEY (BookID) REFERENCES Books(BookID),
25     FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
26 );
27
28 -- Insert data
29 INSERT INTO Books (Title, Author, PublishedYear)
30 VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
31
32 INSERT INTO Members (FirstName, LastName, MembershipDate)
33 VALUES ('Alice', 'Smith', '2020-01-01');
34
35 INSERT INTO Loans (BookID, MemberID, LoanDate)
36 VALUES (1, 1, '2024-06-01');
37
38 -- Retrieve all books currently issued
39 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
40 FROM Loans l
41 JOIN Books b ON l.BookID = b.BookID
42 JOIN Members m ON l.MemberID = m.MemberID
43
44 -- Find overdue books (Loan date > 30 days)
45 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
46 FROM Loans l
47 JOIN Books b ON l.BookID = b.BookID
48 JOIN Members m ON l.MemberID = m.MemberID
49 WHERE DATE(l.LoanDate) < DATE('now', '-30 days');
50
51 -- Count number of books loaned by each member
52 SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
53 FROM Members m
54 LEFT JOIN Loans l ON m.MemberID = l.MemberID
55 GROUP BY m.MemberID;
```

| Title            | FirstName | LastName | LoanDate   |
|------------------|-----------|----------|------------|
| The Great Gatsby | Alice     | Smith    | 2024-06-01 |

```
File Edit Selection View Go Run Terminal Help
-- Assignment-16-1.sql -- AI-16-1.sql -- ass-16.sql --
34 VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
35
36 INSERT INTO Members (FirstName, LastName, MembershipDate)
37 VALUES ('Alice', 'Smith', '2020-01-01');
38
39 INSERT INTO Loans (BookID, MemberID, LoanDate)
40 VALUES (1, 1, '2024-06-01');
41
42 -- Retrieve all books currently issued
43 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
44 FROM Loans l
45 JOIN Books b ON l.BookID = b.BookID
46 JOIN Members m ON l.MemberID = m.MemberID;
47
48 -- Find overdue books (Loan date > 30 days)
49 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
50 FROM Loans l
51 JOIN Books b ON l.BookID = b.BookID
52 JOIN Members m ON l.MemberID = m.MemberID
53 WHERE DATE(l.LoanDate) < DATE('now', '-30 days');
54
55 -- Count number of books loaned by each member
56 SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
57 FROM Members m
58 LEFT JOIN Loans l ON m.MemberID = l.MemberID
59 GROUP BY m.MemberID;
60
```

| Title            | FirstName | LastName | LoanDate   |
|------------------|-----------|----------|------------|
| The Great Gatsby | Alice     | Smith    | 2024-06-01 |

```
File Edit Selection View Go Run Terminal Help
-- Assignment-16-1.sql -- AI-16-1.sql -- ass-16.sql --
34 VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
35
36 INSERT INTO Members (FirstName, LastName, MembershipDate)
37 VALUES ('Alice', 'Smith', '2020-01-01');
38
39 INSERT INTO Loans (BookID, MemberID, LoanDate)
40 VALUES (1, 1, '2024-06-01');
41
42 -- Retrieve all books currently issued
43 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
44 FROM Loans l
45 JOIN Books b ON l.BookID = b.BookID
46 JOIN Members m ON l.MemberID = m.MemberID;
47
48 -- Find overdue books (Loan date > 30 days)
49 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
50 FROM Loans l
51 JOIN Books b ON l.BookID = b.BookID
52 JOIN Members m ON l.MemberID = m.MemberID
53 WHERE DATE(l.LoanDate) < DATE('now', '-30 days');
54
55 -- Count number of books loaned by each member
56 SELECT m.FirstName, m.LastName, COUNT(l.BookID) AS TotalBooks
57 FROM Members m
58 LEFT JOIN Loans l ON m.MemberID = l.MemberID
59 GROUP BY m.MemberID;
60
```

| FirstName | LastName | TotalBooks |
|-----------|----------|------------|
| Alice     | Smith    | 1          |

## OBSERVATION:

This task helped me learn how a library system works using a database. I understood how to track books and members. I also learned how indexing improves performance and how to find overdue books using queries.

## TASK-4:

### PROMPT:

--create tables Users, Products, Orders, OrderDetails.

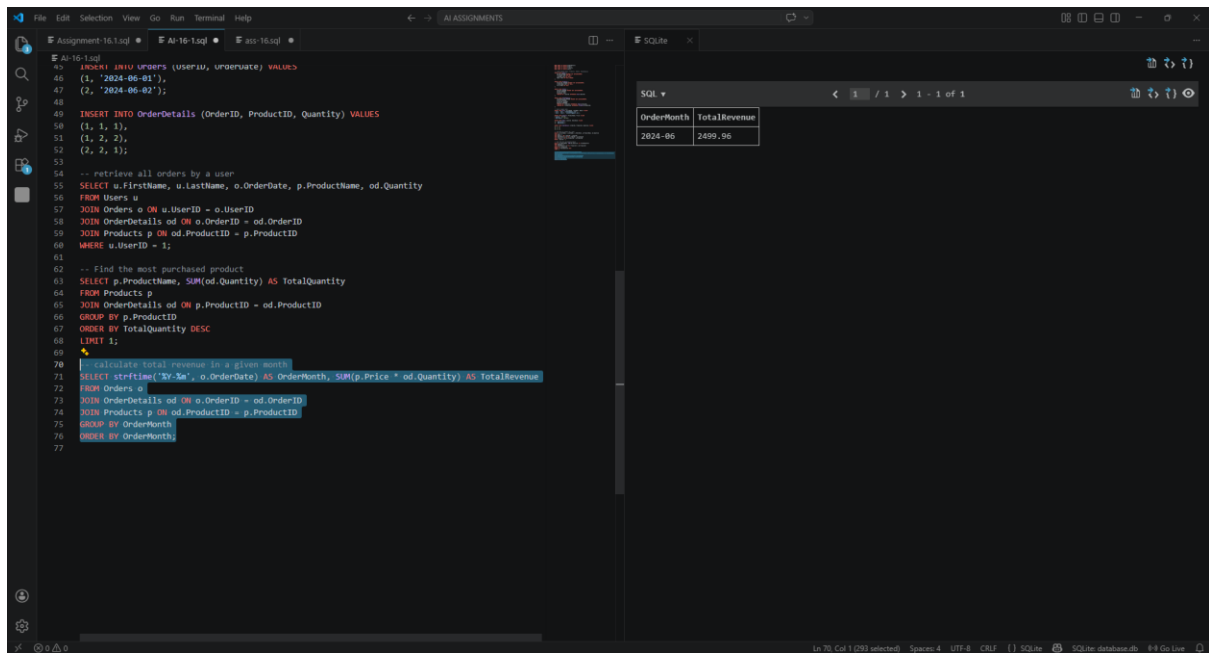
CODE:

```
File Edit Selection View Go Run Terminal Help
Assignment-16.1.sql AI-16-1.sql ass-16.sql SQLite
AI-16-1.sql
39 ('Jane', 'Smith', 'janesmith@gmail.com');
40
41 INSERT INTO Products (ProductName, Price) VALUES
42 ('Laptop', 999.99),
43 ('Smartphone', 499.99);
44
45 INSERT INTO Orders (UserID, OrderDate) VALUES
46 (1, '2024-06-01'),
47 (2, '2024-06-02');
48
49 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
50 (1, 1, 1),
51 (1, 2, 2),
52 (2, 2, 1);
53
54 -- retrieve all orders by a user
55 SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
56 FROM Users u
57 JOIN Orders o ON u.UserID = o.UserID
58 JOIN OrderDetails od ON o.OrderID = od.OrderID
59 JOIN Products p ON od.ProductID = p.ProductID
60 WHERE u.UserID = 1;
61
62 -- Find the most purchased product
63 SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
64 FROM Products p
65 JOIN OrderDetails od ON p.ProductID = od.ProductID
66 GROUP BY p.ProductID
67 ORDER BY TotalQuantity DESC
68 LIMIT 1;
69
70 -- calculate total revenue in a given month
71 SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
72 FROM Orders o
73 JOIN OrderDetails od ON o.OrderID = od.OrderID
74 JOIN Products p ON od.ProductID = p.ProductID
75 GROUP BY OrderMonth;
76 ORDER BY OrderMonth;
77
```

| FirstName | LastName | OrderDate  | ProductName | Quantity |
|-----------|----------|------------|-------------|----------|
| John      | Doe      | 2024-06-01 | Laptop      | 1        |
| John      | Doe      | 2024-06-01 | Smartphone  | 2        |

```
File Edit Selection View Go Run Terminal Help
Assignment-16.1.sql AI-16-1.sql ass-16.sql SQLite
AI-16-1.sql
40 INSERT INTO Users (userid, username) VALUES
41 (1, '2024-06-01'),
42 (2, '2024-06-02');
43
44 INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
45 (1, 1, 1),
46 (1, 2, 2),
47 (2, 2, 1);
48
49 -- retrieve all orders by a user
50 SELECT u.FirstName, u.LastName, o.OrderDate, p.ProductName, od.Quantity
51 FROM Users u
52 JOIN Orders o ON u.UserID = o.UserID
53 JOIN OrderDetails od ON o.OrderID = od.OrderID
54 JOIN Products p ON od.ProductID = p.ProductID
55 WHERE u.UserID = 1;
56
57 -- Find the most purchased product
58 SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity
59 FROM Products p
60 JOIN OrderDetails od ON p.ProductID = od.ProductID
61 GROUP BY p.ProductID
62 ORDER BY TotalQuantity DESC
63 LIMIT 1;
64
65 -- calculate total revenue in a given month
66 SELECT strftime('%Y-%m', o.OrderDate) AS OrderMonth, SUM(p.Price * od.Quantity) AS TotalRevenue
67 FROM Orders o
68 JOIN OrderDetails od ON o.OrderID = od.OrderID
69 JOIN Products p ON od.ProductID = p.ProductID
70 GROUP BY OrderMonth;
71 ORDER BY OrderMonth;
72
```

| ProductName | TotalQuantity |
|-------------|---------------|
| Smartphone  | 3             |



## OBSERVATION:

This task gave me an idea about how e-commerce websites store data. I learned how different tables like users, products, and orders are connected. Writing queries like total revenue helped me understand business analysis.

## TASK-5:

### PROMPT:

--create tables Student,Subjects,Exams,Results.primary keys, foreign keys, referential integrity constraints

### CODE:

The screenshot shows a SQL IDE with a script editor on the left and a query result window on the right. The script includes SQL commands to create tables, insert sample data, and a query to select individual results for a student. The query result window displays a table with the following data:

| FirstName | LastName | SubjectName | Score |
|-----------|----------|-------------|-------|
| John      | Doe      | Mathematics | 85    |
| John      | Doe      | Science     | 90    |

The screenshot shows the same SQL IDE with the same script. The query result window now displays a table with the following data, showing the average scores for each subject:

| SubjectName | AverageScore |
|-------------|--------------|
| Mathematics | 80.0         |
| Science     | 89.0         |

## OBSERVATION:

This task helped me understand how exam data is managed. I learned how to store marks and calculate averages using SQL. It also improved my skills in using joins and aggregation functions.